

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Cryptology – Lab_7

Reg. Number	Name	Branch
261100690017	**Shravya B**	**Cybersecurity**

03/11/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

1. Write Python code to perform both encryption and decryption)
   a. columnar transposition

main.py

```python
mode = input("Encrypt or decrypt (e/d): ").lower()
text = input("Enter text: ").upper().replace(" ", "")
key = input("Enter key (no repeated letters): ").upper()

if mode not in ("e", "d"):
    print("Mode must be e or d!")
    exit()
if not text.isalpha() or not key.isalpha():
    print("Text and key must be letters only!")
    exit()
if len(key) < 2 or len(set(key)) != len(key):
    print("Key needs 2+ letters with no repeats!")
    exit()

n = len(key)
result = ""

if mode == "e":
    while len(text) % n != 0:
        text += "X"                     # fill the last row
    for letter in sorted(key):          # columns in alphabetical order
        col = key.index(letter)
        for i in range(col, len(text), n):
            result += text[i]
else:
    if len(text) % n != 0:
        print("Ciphertext length must be a multiple of the key length!")
        exit()
    rows = len(text) // n
    columns = [""] * n
    pos = 0
    for letter in sorted(key):
        col = key.index(letter)
        columns[col] = text[pos:pos + rows]
        pos += rows
    for r in range(rows):
        for c in range(n):
            result += columns[c][r]

print("Result:", result)
```

Output1:
Encrypt or decrypt (e/d): e
Enter text: Hello
Enter key (no repeated letters): o
Key needs 2+ letters with no repeats!

Output2:
Encrypt or decrypt (e/d): d
Enter text: Khoor
Enter key (no repeated letters): h
Key needs 2+ letters with no repeats!

   b. double columnar transposition:

```
c. def encrypt(text, key):
d.     n = len(key)
e.     while len(text) % n != 0:
f.         text += "X"
g.     result = ""
h.     for letter in sorted(key):
i.         col = key.index(letter)
j.         for i in range(col, len(text), n):
k.             result += text[i]
l.     return result
m.
n. def decrypt(text, key):
o.     n = len(key)
p.     rows = len(text) // n
q.     columns = [""] * n
r.     pos = 0
s.     for letter in sorted(key):
t.         col = key.index(letter)
u.         columns[col] = text[pos:pos + rows]
v.         pos += rows
w.     result = ""
x.     for r in range(rows):
y.         for c in range(n):
z.             result += columns[c][r]
aa.    return result
bb.
cc.mode = input("Encrypt or decrypt (e/d): ").lower()
dd.text = input("Enter text: ").upper().replace(" ", "")
ee.key1 = input("Enter key 1 (no repeated letters): ").upper()
ff.key2 = input("Enter key 2 (no repeated letters): ").upper()
gg.
hh.if mode not in ("e", "d"):
ii.    print("Mode must be e or d!")
jj.    exit()
kk.if not text.isalpha():
ll.    print("Text must be letters only!")
mm.    exit()
nn.for key in (key1, key2):
oo.    if not key.isalpha() or len(key) < 2 or len(set(key)) !=
   len(key):
pp.        print("Each key needs 2+ letters with no repeats!")
qq.        exit()
rr.
ss.if mode == "e":
tt.    print("Result:", encrypt(encrypt(text, key1), key2))
uu.else:
vv.    if len(text) % len(key2) != 0:
```

```
ww.         print("Ciphertext length must be a multiple of key 2
   length!")
xx.         exit()
yy.     step = decrypt(text, key2)            # undo round 2 first
zz.     if len(step) % len(key1) != 0:
aaa.             print("Ciphertext length does not fit key 1!")
bbb.             exit()
ccc.          print("Result:", decrypt(step, key1)) # then undo round
   1
```

Output1:

Encrypt or decrypt (e/d): e

Enter text: Hello

Enter key 1 (no repeated letters): h

Enter key 2 (no repeated letters): o

Each key needs 2+ letters with no repeats!

Output2:

Encrypt or decrypt (e/d): d

Enter text: Khoor

Enter key 1 (no repeated letters): K

Enter key 2 (no repeated letters): R

Each key needs 2+ letters with no repeats!

2.Explore the OpenSSL tool

Aim

To explore the OpenSSL tool and understand its applications in cryptography, encryption, decryption, hashing, digital certificates, and secure communication.

Introduction

OpenSSL is an open-source cryptographic toolkit used to implement Secure Sockets Layer (SSL) and Transport Layer Security (TLS) protocols. It provides various cryptographic functions to protect data from unauthorized access and modification.

OpenSSL is widely used by cybersecurity professionals, system administrators, and developers for encryption, decryption, hashing, public and private key generation, digital certificate management, and secure communication.

Features of OpenSSL

1. **Encryption and Decryption:** Protects confidential information by converting plaintext into ciphertext and recovering the original data.

2. **Hashing:** Generates fixed-length hash values to verify data integrity.
3. **Key Generation:** Generates public and private cryptographic keys.
4. **Digital Certificates:** Creates and examines digital certificates.
5. **SSL/TLS Communication:** Establishes and examines secure connections between clients and servers.
6. **Random Number Generation:** Generates cryptographically secure random values.

4. Exploring OpenSSL Commands

Experiment 1: Check OpenSSL Version

Command:

openssl version

Explanation: Displays the installed version of OpenSSL.

Observation: The OpenSSL version is displayed, confirming that the tool is installed and working.

Experiment 2: Generate Random Numbers

Command:

openssl rand -hex 16

Explanation:

- rand generates random bytes.
- -hex displays the output in hexadecimal format.
- 16 specifies the number of bytes to generate.

Observation: OpenSSL successfully generates random hexadecimal data that can be used in cryptographic applications.

Experiment 3: Generate SHA-256 Hash

Step 1: Create a sample file.

echo "Hello Cybersecurity" > sample.txt

Step 2: Generate its hash.

openssl dgst -sha256 sample.txt

Explanation: The SHA-256 algorithm generates a 256-bit hash value from the contents of the file.

Observation: A unique hexadecimal hash value is generated. If the file content changes, the hash value also changes.

Conclusion: SHA-256 is useful for checking file integrity and detecting unauthorized modifications.

Experiment 4: Encrypt and Decrypt a File Using AES-256

Step 1: Create a plaintext file.

echo "This is a confidential cybersecurity message." > plaintext.txt

Step 2: Encrypt the file.

openssl enc -aes-256-cbc -salt -pbkdf2 -in plaintext.txt -out encrypted.txt

Enter a password when prompted.

Step 3: View the encrypted file.

cat encrypted.txt

The contents appear unreadable because the file has been encrypted.

Step 4: Decrypt the file.

openssl enc -d -aes-256-cbc -pbkdf2 -in encrypted.txt -out decrypted.txt
Enter the same password used during encryption.
Step 5: Display the decrypted content.
cat decrypted.txt
Expected Output:
This is a confidential cybersecurity message.
Observation: The original plaintext is successfully recovered after decryption.
Conclusion: AES-256 is a symmetric encryption algorithm that uses the same secret password-derived key for encryption and decryption.
Experiment 5: Generate RSA Public and Private Keys
Step 1: Generate an RSA private key.
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
Step 2: Generate the corresponding public key.
openssl pkey -in private_key.pem -pubout -out public_key.pem
Step 3: Display the generated files.
ls -l *.pem
Step 4: Examine the public key.
openssl pkey -pubin -in public_key.pem -text -noout
Observation: OpenSSL successfully generates an RSA public-private key pair.
Conclusion: RSA is an asymmetric cryptographic algorithm that uses separate public and private keys for cryptographic operations.
Experiment 6: Generate a Self-Signed Digital Certificate
Step 1: Generate a self-signed certificate.
openssl req -new -x509 -key private_key.pem -out certificate.crt -days 365
Enter the requested certificate details, such as country, state, organization, and common name. Use localhost as the common name for this local experiment.
Step 2: Display certificate information.
openssl x509 -in certificate.crt -text -noout
Step 3: Check the certificate validity period.
openssl x509 -in certificate.crt -noout -dates
Observation: The certificate displays information such as the subject, issuer, public key, validity period, and digital signature.
Conclusion: Digital certificates associate identities with public keys and are used in secure communication.
Experiment 7: Examine an SSL/TLS Connection
Command:
openssl s_client -connect example.com:443 -servername example.com
Explanation:

- s_client starts an SSL/TLS client.
- -connect specifies the server address and port.

- -servername specifies the hostname using Server Name Indication (SNI).

Observation: OpenSSL displays the server certificate, TLS protocol version, cipher suite, and connection information.

Conclusion: The command helps examine secure communication between a client and a server.

5. Applications of OpenSSL

- Protecting confidential files through encryption.
- Verifying file integrity using cryptographic hashes.
- Generating public and private cryptographic keys.
- Creating and managing digital certificates.
- Testing secure SSL/TLS connections.
- Supporting secure communication over computer networks.

6. Security Precautions

- Private keys must be kept confidential.
- Strong passwords should be used to protect encrypted keys.
- Secure hashing algorithms such as SHA-256 should be preferred over outdated algorithms such as MD5 and SHA-1.
- Encryption passwords should never be shared with unauthorized individuals.
- Digital certificates should be verified before being trusted.
- Experiments should only be performed on systems for which authorization has been obtained.

7. Result

The OpenSSL tool was successfully explored. Different cryptographic operations, including random number generation, SHA-256 hashing, AES-256 encryption and decryption, RSA key generation, digital certificate creation, and SSL/TLS connection testing, were studied and performed.

OpenSSL is an important cryptographic tool used in cybersecurity to protect sensitive information and secure network communication. Through this practical exercise, the working of symmetric encryption, asymmetric encryption, hashing, digital certificates, and SSL/TLS communication was understood. The experiment provided practical knowledge of how cryptographic techniques are applied in real-world security systems.